PRD · 2026-ERA TEMPLATE

Governed AI CMS

An Arabic newsroom assistant that turns one topic line into a schema-validated, publish-ready CMS entry — with deterministic guardrails, async AI critique, and a human-override record on every edit.

Owner Ram **Status** Working prototype, local pilot **Repo** arabic-ai-cms **Updated** 2026-10-11

01 The Decision This Records NEW

We're shipping a **deterministic-first** generation pipeline as the default — fast, rule-validated single-pass drafts — with a slower **3-agent critique-and-refine** mode as an opt-in, because editors need a trustworthy fast path for routine articles and a visibly-better second opinion only when it's worth the extra 3×latency and cost.

02 Purpose TIMELESS

What	A tool that takes a one-line Arabic topic and returns a complete, CMS-ready article unit — 3 headline options, a URL slug, SEO title and description, a 400–550 word Modern Standard Arabic body, 5 tags, and a summary — already checked against a fixed schema before an editor ever sees it.
Who it's for	The Arabic CMS editor who currently hand-writes headlines, slugs, meta description and tags for every piece — a mechanical pass that happens on top of the actual writing, every single time.
Why it matters	That metadata pass costs an editor roughly 15–20 minutes per article and is where fields get skipped under deadline pressure — missing tags, a slug that's just transliterated English, an SEO description nobody timed.

Hypothesis

We believe generating a schema-enforced first draft, gated by deterministic checks and backed by an optional AI critique-and-refine pass, will cut per-article prep time by 70%+ while keeping the human edit-delta on finalized drafts low enough that editors trust the draft rather than rewrite it.

03 The Prototype NEW

Running locally, not yet deployed to a shared URL:

- `http://localhost:5001` — Express API + static client, no auth layer yet

04 Features & How They Work TIMELESS

SIMPLE MODE — DEFAULT PATH

`topic input` → `POST /api/generate` → `prompt built` → `Gemini call` → `schema + word-count validate` → `retry once on fail` → `deterministic score` → `draft returned`

The editor sees the draft immediately. In the background, an async AI critique call scores the same draft 1–10 with Arabic feedback and writes it to the audit log — it never blocks or delays the response, and a failed critique just marks the record `FAILED` rather than surfacing an error.

Editing the body and pressing **Finalize & Save** runs a word-level diff between the original and the edited text and stores the resulting **override percentage** — the system's own measure of how much a human had to change.

AGENTIC MODE — OPT-IN, 3 AGENTS

🤖 `generator → draft v1` → 🔍 `critic → scored feedback` → ✨ `refiner → draft v2` → `editor picks v1 or v2`

The UI shows each agent's status live, then a side-by-side comparison of draft v1 and v2 with the critic's per-field feedback (headlines, body, SEO, tags, language) and the per-stage timing. Whichever version the editor picks is finalized the same way as simple mode.

BEHAVIOUR CONTRACT — AI FEATURES

- **How it responds:** JSON only, no markdown, no explanation, Modern Standard Arabic only, with every field present even when short.
- **What it must never do:** return a different field shape than specified, mix in non-Arabic text, or silently truncate the body outside the accepted 350–650 word range.
- **Fallback when unsure:** on malformed JSON, a wrong field count, or a body outside the accepted word range, the system retries the call exactly once. A second failure throws rather than shipping a malformed draft to an editor.
- **Tone and limits:** newsroom-neutral MSA; slugs are restricted to Arabic characters and hyphens only; tags are deduplicated and normalized before display.

05 Success Measurement & Evals UPGRADED

Metric to graduate	≥ 90% of simple-mode generations pass schema and word-count validation on the first attempt (`retryTriggered: false`), and the average override percentage on finalized drafts stays under 25% — measured over the first 30 days of pilot use.
Eval set	20 real editorial topics across Politics, Economy, Technology and Sports — committed at `server/evals/eval-set.json` , each paired with the expected field shape (3 headlines, 5 tags, Arabic-hyphen slug, 350–650 word body, ≤160-char SEO description). Run it with `node server/evals/run-evals.js` against any running instance — pass/fail is checked by the script, not by an editor.

SERVER/EVALS/EVAL-SET.JSON — SAMPLE ROWS

#	CATEGORY	INPUT
01	Politics	اجتماع وزراء الخارجية الخليجيين لبحث التعاون الأمني المشترك
06	Economy	ارتفاع مؤشر سوق دبي المالي بدعم من قطاع العقارات

#	CATEGORY	INPUT
11	Technology	إطلاق مبادرة وطنية لتسريع تبني تقنيات الذكاء الاصطناعي
16	Sports	المنتخب الوطني يستعد لمواجهة حاسمة في تصفيات كأس العالم

5 cases per category, 20 total. Full list and the runnable checker live in the repo.

CHECK	RULE	WEIGHT
Word count	`400–550 words`	+3
Keyword presence	`input's first word appears in body`	+2
Repetition	`unique/total sentence ratio > 0.7`	+3
SEO title	`present, > 10 characters`	+2

This deterministic score (max 10) runs on every simple-mode draft and is what the UI's `Quality` badge shows. It's separate from the async AI critique score, which judges the same draft on editorial grounds in the background.

06 Scope & Non-Goals MERGED

IN

- MSA article generation: headlines, slug, SEO fields, body, tags, summary
- Deterministic schema + word-count validation with one retry
- Async AI critique and audit trail
- 3-agent generate → critique → refine mode
- Human edit-delta tracking on finalize
- SQLite generation log

OUT

- Direct publish-to-CMS integration
- Non-Arabic languages or dialectal Arabic
- Image generation
- Auth, roles, or multi-user review workflow
- Scheduling or states beyond "finalized"

Trade-offs accepted: a JAIS fallback model path is wired in `jaisService.js` but not load-bearing — Gemini is the only active provider in v1. A single SQLite file is fine at prototype scale but won't hold up to concurrent writers. The API has no rate-limiting or auth, acceptable for a local/internal pilot and nothing wider.

07 Rollout & Rollback NEW

A small %, not everyone at once: one editor, real but non-time-critical articles, two weeks, before it reaches the wider newsroom. Simple mode is the default so nobody is defaulted into the slower, costlier agentic path.

Kill switch: blanking `GEMINI_API_KEY` or flipping `AGENT_PROVIDER` away from `gemini` in `.env` stops every live model call immediately — both call sites (`modelService.js` , `critiqueService.js`) read from the same env var, so it's a single point of control.

The number that pulls it: if the daily average override percentage climbs past 50% — editors effectively rewriting, not editing — the pilot pauses and editors revert to drafting manually while the prompt or model is revisited.

08 Release Criteria & Change Log TIMELESS

Definition of done: a topic submitted in simple mode returns a schema-valid 400–550 word MSA draft, an editor can edit and finalize it with a correctly computed override percentage, end to end, with zero unhandled exceptions across the 20-item eval set. `Testable` — not "feels good."

v1 · earlier

Initial pipeline

Gemini-backed single and agentic generation, SQLite logging, override tracking shipped.

2026-10-08

Billing restored, models updated PASS

Confirmed `GEMINI_API_KEY` live after billing was enabled. `gemini-2.5-flash` / `gemini-2.5-pro` had been retired for this tier; repointed all three call sites (`modelService.js` , `critiqueService.js` , `jaisService.js`) to `gemini-3.8-flash` and verified end-to-end generation.

2026-10-08

Loading indicator added

Simple mode's "Generating…" text now has a spinner (`index.html` , `style.css`); agentic mode already had its own step icons.

2026-10-11

Quality badge bug fixed was FAIL

Client read `meta.qualityScore` , which the API never sends — it sends `meta.deterministicScore` . Badge always showed "undefined/10 — Needs Review" regardless of the real score. Fixed in `client/app.js` .

THE GOLDEN STANDARD · 2026

Could a developer **or an AI agent** build this feature from only this document, without asking a single question? Section 5's eval set is now a committed, runnable file — that gap is closed. **Section 3** still has no public prototype link, only a local one. That's the one thing standing between this draft and "done."
